Detecting Malaria In Image Cells

François Bryan

***Abstract*—This report presents an analysis and improvement of a Kaggle project focused on malaria cell image classification using deep learning techniques. The objective is to design a convolutional neural network capable of distinguishing infected from uninfected blood cells with high reliability. After reviewing a previous implementation, methodological and architectural refinements were introduced to enhance robustness and predictive performance.**

I. Introduction

Within the framework of the course PROJ-H419, students are required to design and implement a deep learning solution applied to a biomedical image dataset. In this context, the project entitled "Detecting Malaria in Cell Images"[1] was selected.

Malaria is a globally widespread infectious disease transmitted to humans through the bite of infected mosquitoes. The parasite initially migrates to the liver, where it develops over a period ranging from several days to a few weeks. Once mature, it infects red blood cells. Clinical symptoms generally appear during this blood stage and may include fever, headaches, respiratory distress, organ failure, coma, and, in severe cases, death. [2][3]

Transmission mainly occurs through mosquito vectors, but direct blood contact may also spread the disease (e.g., mother-to-child transmission, contaminated needles, or blood transfusions). Malaria is particularly prevalent in tropical and subtropical regions such as Sub-Saharan Africa, Central America, and parts of the Pacific.[2][3]

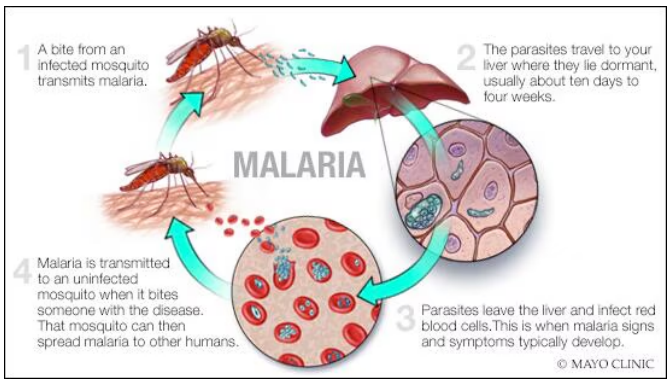

Fig. 1. Malaria Infection Process [3]

With millions of cases reported annually and significant mortality among vulnerable populations, early detection is essential to ensure rapid treatment. [2][3]A dataset containing 27,558 labeled images of infected and uninfected blood cells is available on Kaggle. The objective of this project is to develop a model capable of reliably classifying these cell images.[1]

II. Related Work

The initial phase of the analysis consisted of studying a convolutional neural network (CNN) implementation published approximately seven years ago on Kaggle.[4]

In this approach, RGB cell images were first loaded from the infected and uninfected directories. Each image was resized to 50×50 pixels to reduce computational complexity and standardize the input dimensions. To increase dataset variability and improve generalization, data augmentation techniques were applied. Specifically, rotated versions of each image (45° and 75°) and blurred variants were generated. Each augmented sample retained its corresponding label (infected or uninfected), and all samples were merged into a single dataset.

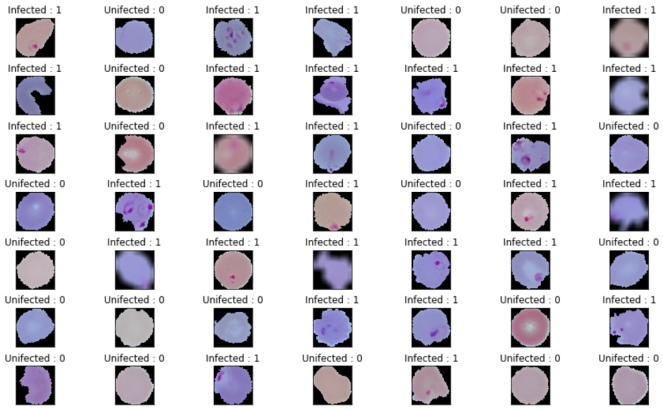

Fig. 2. Example of Data Augmentation

Through augmentation, the dataset size increased from 27,558 to 96,453 images.

The dataset was then divided into training and testing sets, with 20% reserved for testing and validation and 80% for training. The training set was further validate with the validation subset to monitor overfitting and optimize hyperparameters.

```
train data shape (77162, 50, 50, 3) ,eval data shape (9645, 50, 50, 3)
, test data shape (9646, 50, 50, 3)
```

Fig. 3. Dataset Splits

A preliminary inspection of the class distribution reveals a slight imbalance between infected and uninfected samples.

A. Detailed CNN Architecture

The implemented model follows a sequential convolutional neural network architecture. The input images are first reshaped into a four-dimensional tensor of shape

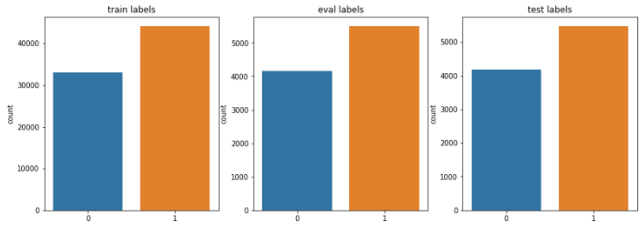

Fig. 4. Class Distribution Across Splits

$(batch, 50, 50, 3)$ in order to be compatible with the convolutional layers, which require structured spatial input.

The feature extraction stage consists of four convolutional layers. The first three convolutional layers apply different kernel sizes and numbers of filters while using the same activation function (ReLU). The depth of the network evolves from 50 feature maps in the first layer to 90 in the second layer, before decreasing to 10 in the third layer.

Two padding strategies are employed. The `same` padding preserves spatial dimensions by adding zero-padding around the borders of the image. In contrast, the `valid` padding applies convolution only where the kernel fully overlaps the input, thereby reducing spatial dimensions.

The ReLU activation function is used after each convolutional layer to introduce non-linearity, enabling the network to learn complex and non-linear decision boundaries.

A first max-pooling operation is then applied to reduce spatial resolution and aggregate the most salient features. This operation decreases computational complexity while retaining the strongest activations. A fourth convolutional layer, containing 5 filters, is subsequently applied. This final convolutional stage produces a compact set of high-level discriminative feature maps. A second max-pooling layer further reduces spatial dimensions.

B. Fully Connected Classifier

After the convolutional blocks, the feature maps are flattened into a one-dimensional vector. This flattened representation serves as input to the fully connected classifier.

The classifier is composed of three dense layers with progressively decreasing dimensionality: 2000 units, followed by 1000 units, and finally 500 units. Each dense layer uses the ReLU activation function.

The final output layer projects the representation onto two neurons corresponding to the binary classification task: infected versus uninfected. The output consists of raw logits. These logits represent unnormalized scores. A softmax function is then applied to transform these scores into a probability distribution over the classes. The predicted class is obtained by selecting the index of the maximum probability using the argmax operation.

III. My Implementation

In this section, I describe the modifications and improvements I applied to the original Kaggle implementation.

Since the original code was published seven years ago, several parts were outdated and no longer compatible with modern frameworks. My primary goal was to update the implementation according to current standards while introducing incremental improvements where feasible.

A. Resizing Image Cells

The images in the dataset vary in size. An initial analysis revealed that the average cell image has dimensions of approximately 132×132 pixels (Figure fig.5).

```
Parasitized
Uninfected
------------------------------
Total images : 27558
Min width : 46
Min height : 40
Max width : 394
Max height : 385
Average size : 132 x 132
```

Fig. 5. Distribution of Original Image Sizes

Reducing the images to 50×50 pixels, as in the original implementation, could lead to information loss. Therefore, I decided to evaluate the performance of two configurations: one with images resized to 50×50 and another with images resized to 128×128, in order to assess the impact of input resolution on model performance.

B. Data Augmentation

Modern data augmentation libraries offer a wider range of transformations than those available seven years ago. For this purpose, I chose the "Albumentations" library [5], which provides a rich palette of augmentation techniques.

To reproduce the original approach, I implemented an augmentation pipeline for 50×50 images that includes:

- Rotations up to $75°$ with a probability of 50%,
- Gaussian blur with a probability of 50%,
- Normalization of pixel values to $[0, 1]$ for training.

Validation images are only resized and normalized.

For my new pipeline, I resize them in higher-resolution images (128×128). More, I implemented a more extensive augmentation strategy to increase variability while in the same time not overloading the model:

- Horizontal and vertical flips with a probability of 50%
- Rotations up to $180°$ with a probability of 70%,
- Random brightness and contrast adjustments with a probability of 50%,
- Hue and saturation shifts with a probability of 30%,
- Gaussian blur with a probability of 20%.

The probability percentage does not follow a particular rule.

Validation images are only resized to 128×128 and normalized.

C. Data Generator

In the original implementation, each augmented image was stored explicitly. This approach significantly increases storage requirements, and it would have required even more space with my extended augmentation pipeline, which includes a larger number of transformations.

To address this limitation, a ”DataGenerator” class is commonly used. Instead of precomputing and storing all augmented samples, the generator dynamically loads images batch by batch during training. For each batch, the selected images are read from disk, the transformations are applied according to their associated probabilities, and the model performs forward and backward propagation to update its weights before moving to the next batch.

In practice, only one batch is processed and transformed at a time, which considerably reduces memory usage. Moreover, since the transformations are stochastic, the same image can undergo different augmentations across epochs. This results in a virtually infinite and more diverse training set, improving generalization while maintaining efficient memory management.

D. Data Splits

I applied an 80-20 split for training and validation to have more data for training. Due to random shuffling, the distribution of infected and uninfected cells varies slightly across subsets. Figures fig.6 and fig.7 illustrate the distributions and counts of each class in the splits.

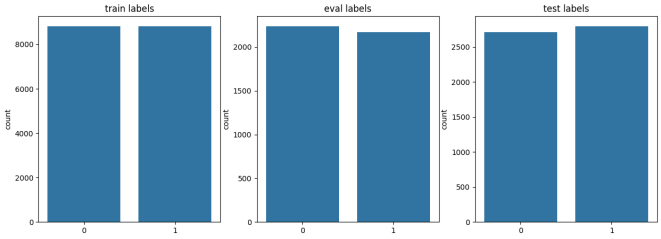

Fig. 6. Distribution of Classes Across Splits

```
train data shape (17636,) ,eval data shape (4410,) , test data shape (5512,)
```

Fig. 7. Number of Cells per Split

E. Model Architecture

I implemented a simplified convolutional neural network using the same kernel size (3,3) and padding (’same’) for all convolutional layers. The depth of the layers varies, following modern conventions, with choices of 32, 64, or 128 filters per layer. They aslo all use the same activation function (’Relu’).

The fully connected classifier consists of dense layers with 64 and 128 units. Dropout with a rate of 0.5 was applied after each dense layer to reduce overfitting during training. Each dense layers use the activation function ’Relu’ and the last one the ’sigmoid’ function. The model is optimize following the optimizer ’Adam’(Adaptive Moment Estimation) that adapts the learning rate to converge fast.[6] The loss function is the binary Cross Entropy. It is principally use for binary classification as we are in. It tries to maximize the model confidence (probability of predict good label) in its binary choice.[7]

F. Objective

My first objective was to identify if the using an higher resolution gives better results than lower resolution.

The second objective was to find the best configuration of convolutional and dense layers within feasible computational limits. To this end, I used TensorBoard to systematically test different numbers of convolutional and dense layers and various sizes of nodes, evaluating their impact on training and validation performance (inspired from [8].

IV. Results & Discussion

A. First Objective: Evaluating Image Resolution

To address the first objective, it was necessary to evaluate both resizing approaches (50×50 and 128×128) under two distinct conditions: the original author’s baseline augmentation and the newly implemented extended augmentation pipeline.

To ensure computational efficiency during this comparative phase, a lightweight baseline model was utilized. This model consisted of three convolutional layers (each with a constant depth of 32 filters) separated by max-pooling layers, followed by a single dense layer of 64 nodes. A cross-validation strategy was applied across the entire training dataset to robustly compare the two resizing methods.

```
Average Validation Accuracy (50x50): 0.9578 ± 0.0040
Average Validation Loss (50x50): 0.1258 ± 0.0099
Average Validation Accuracy (128x128): 0.9579 ± 0.0024
Average Validation Loss (128x128): 0.1358 ± 0.0081
```

Fig. 8. Models Performance with original features

```
Average Validation Accuracy (50x50): 0.9555 ± 0.0025
Average Validation Loss (50x50): 0.1294 ± 0.0115
Average Validation Accuracy (128x128): 0.9563 ± 0.0020
Average Validation Loss (128x128): 0.1350 ± 0.0100
```

Fig. 9. Models Performance with augmented features

The results indicate that under the original, more restricted augmentation pipeline, the performance difference between the two resolutions is negligible, offering little insight into an optimal size. However, when subjected to the newly extended augmentation pipeline, a slight performance divergence emerges. The 128×128 resolution achieves a marginally higher accuracy (an increase of 0.0012).

From these observations, it can be deduced that higher-resolution images help the network retain crucial morphological details. This retention becomes particularly important when the input data is subjected to heavier and more varied

perturbations, as the larger format prevents those critical details from being distorted or lost.

B. Second Objective: Architecture Optimization

The second objective aimed to identify the optimal combination of convolutional and dense layers to maximize predictive performance. Due to the vast number of possible model permutations, cross-validation was not performed during this phase. Training hundreds of models would have been prohibitively expensive in terms of both computational power and time. Instead, a standard validation split was relied upon to track performance.

1) Number of Convolutional Layers: To isolate the impact of the feature extraction stage and maintain time efficiency, initial models were generated without fully connected dense layers. Architectures containing up to three convolutional layers were tested across every possible combination of channel depths. An attempt was made to test four layers, but the training time proved excessive.

In total, 39 models were tested. The results clearly indicated that architectures with three convolutional layers consistently performed the best. Only a single two-layer configuration outperformed the lower-end three-layer models, but it ultimately failed to surpass the best overall three-layer architecture (see Fig. 10, Fig. 11, Fig. 12, and Fig. 13).

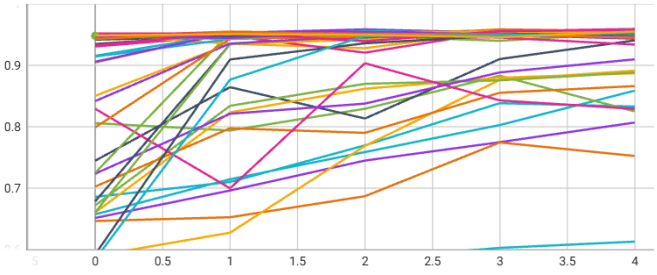

Fig. 10. Models accuracy performance on Validation

2) Number of Dense Layers: Having established that three convolutional layers provide the best baseline feature extraction, the focus shifted to optimizing the classifier head. The purely convolutional models (with zero dense layers) were compared against new configurations utilizing one or two dense layers, with node counts of either 64 or 128. In total, 162 unique model configurations were evaluated.

The results show that the highest-performing models incorporate at least one dense layer. Interestingly, observation of the data reveals that simply adding dense layers does not universally guarantee improved results. For instance, the purely convolutional baseline model (”Conv 3-(128, 128, 128)-Dense 0”) achieved an accuracy of 0.9551 and a loss of 0.1402. In stark contrast, appending a suboptimal dense configuration (”Conv 3-(128, 128, 128)-Dense 1-(64,)”) drastically degraded performance, yielding an accuracy of 0.5079 and a loss of 0.6931.

Run	Value ↓	Step	Relative
model Conv 3-(32, 128, 128)-Dense 0 (0)—1772162345\validation	0,9592	4	9.012 min
model Conv 3-(64, 32, 128)-Dense 0 (0)—1772164102\validation	0,9583	4	7.752 min
model Conv 3-(128, 64, 64)-Dense 0—1772220497\validation	0,9578	4	18.77 min
model Conv 3-(32, 64, 128)-Dense 0 (0)—1772160730\validation	0,9571	4	6.136 min
model Conv 3-(32, 32, 64)-Dense 0 (0)—1772159208\validation	0,9569	4	4.541 min
model Conv 3-(32, 64, 32)-Dense 0 (0)—1772159923\validation	0,9558	4	5.242 min
model Conv 3-(64, 32, 64)-Dense 0 (0)—1772163554\validation	0,9558	4	7.274 min
model Conv 3-(128, 32, 32)-Dense 0 (0)—1772169632\validation	0,9556	4	14.11 min
model Conv 3-(128, 128, 128)-Dense 0—1772226910\validation	0,9551	4	17.75 hr
model Conv 3-(32, 64, 64)-Dense 0 (0)—1772160318\validation	0,9549	4	5.477 min
model Conv 3-(32, 128, 64)-Dense 0 (0)—1772161748\validation	0,9542	4	7.93 min
model Conv 3-(32, 128, 32)-Dense 0 (0)—1772161192\validation	0,9528	4	7.39 min
model Conv 3-(64, 64, 32)-Dense 0 (0)—1772164685\validation	0,9528	4	8.652 min
model Conv 3-(128, 128, 64)-Dense 0—1772225148\validation	0,9524	4	23.27 min
model Conv 3-(64, 32, 32)-Dense 0 (0)—1772163022\validation	0,9515	4	7.064 min
model Conv 3-(32, 32, 32)-Dense 0 (0)—1772158879\validation	0,9512	4	4.376 min
model Conv 3-(128, 64, 32)-Dense 0—1772219093\validation	0,951	4	18.19 min
model Conv 3-(64, 64, 128)-Dense 0 (0)—1772166009\validation	0,9501	4	9.65 min
model Conv 3-(128, 32, 128)-Dense 0 (0)—1772171765\validation	0,9494	4	14.73 min
model Conv 3-(32, 32, 128)-Dense 0 (0)—1772159551\validation	0,949	4	4.946 min
model Conv 3-(64, 128, 128)-Dense 0 (0)—1772168591\validation	0,9481	4	13.86 min
model Conv 3-(64, 64, 64)-Dense 0 (0)—1772165337\validation	0,9481	4	8.924 min
model Conv 3-(128, 32, 128)-Dense 0—1772218713\validation	0,9478	0	0
model Conv 3-(128, 64, 32)-Dense 0 (0)—1772172872\validation	0,9469	4	16.94 min
model Conv 3-(128, 64, 128)-Dense 0—1772221894\validation	0,9463	4	19.27 min
model Conv 3-(128, 32, 64)-Dense 0 (0)—1772170692\validation	0,9451	4	14.27 min
model Conv 3-(128, 128, 32)-Dense 0—1772223358\validation	0,9449	4	24.05 min
model Conv 3-(64, 128, 64)-Dense 0 (0)—1772167640\validation	0,9433	4	12.66 min
model Conv 2-(32, 128)-Dense 0 (0)—1772152432\validation	0,9406	4	7.007 min
model Conv 3-(64, 128, 32)-Dense 0 (0)—1772166734\validation	0,9338	4	12.07 min
model Conv 2-(64, 64)-Dense 0 (0)—1772153466\validation	0,9098	4	8.301 min
model Conv 2-(128, 64)-Dense 0 (0)—1772155984\validation	0,8914	4	16.41 min

Fig. 11. Models accuracy leader-board

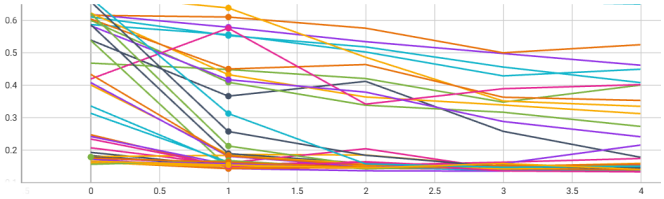

Fig. 12. Models Loss performance on Validation

Run	Value ↑	Step	Relative
model Conv 3-(128, 64, 64)-Dense 0—1772220497\validation	0,133	4	18.77 min
model Conv 3-(32, 128, 64)-Dense 0 (0)—1772161748\validation	0,1353	4	7.93 min
model Conv 3-(32, 64, 128)-Dense 0 (0)—1772160730\validation	0,1364	4	6.136 min
model Conv 3-(64, 32, 128)-Dense 0 (0)—1772164102\validation	0,1372	4	7.752 min
model Conv 3-(32, 32, 64)-Dense 0 (0)—1772159208\validation	0,1385	4	4.541 min
model Conv 3-(128, 32, 32)-Dense 0 (0)—1772169632\validation	0,1385	4	14.11 min
model Conv 3-(64, 32, 32)-Dense 0 (0)—1772163022\validation	0,1387	4	7.064 min
model Conv 3-(32, 64, 32)-Dense 0 (0)—1772159923\validation	0,139	4	5.242 min
model Conv 3-(64, 32, 64)-Dense 0 (0)—1772163554\validation	0,1392	4	7.274 min
model Conv 3-(128, 128, 128)-Dense 0—1772226910\validation	0,1402	4	17.75 hr
model Conv 3-(128, 32, 128)-Dense 0 (0)—1772171765\validation	0,1408	4	14.73 min
model Conv 3-(32, 64, 64)-Dense 0 (0)—1772160318\validation	0,1409	4	5.477 min
model Conv 3-(64, 128, 128)-Dense 0 (0)—1772168591\validation	0,1423	4	13.86 min
model Conv 3-(32, 32, 32)-Dense 0 (0)—1772158879\validation	0,1424	4	4.376 min
model Conv 3-(32, 128, 32)-Dense 0 (0)—1772161192\validation	0,1427	4	7.39 min
model Conv 3-(32, 32, 128)-Dense 0 (0)—1772159551\validation	0,1436	4	4.946 min
model Conv 3-(64, 64, 128)-Dense 0 (0)—1772166009\validation	0,1436	4	9.65 min
model Conv 3-(128, 64, 32)-Dense 0—1772219093\validation	0,1465	4	18.19 min
model Conv 3-(64, 64, 32)-Dense 0 (0)—1772164685\validation	0,1486	4	8.652 min
model Conv 3-(32, 128, 128)-Dense 0 (0)—1772162345\validation	0,1486	4	9.012 min
model Conv 3-(128, 128, 64)-Dense 0—1772225148\validation	0,1499	4	23.27 min
model Conv 3-(64, 64, 64)-Dense 0 (0)—1772165337\validation	0,1505	4	8.924 min
model Conv 3-(128, 64, 32)-Dense 0 (0)—1772172872\validation	0,1548	4	16.94 min
model Conv 3-(64, 128, 64)-Dense 0 (0)—1772167640\validation	0,1554	4	12.66 min
model Conv 3-(128, 128, 32)-Dense 0—1772223358\validation	0,1567	4	24.05 min
model Conv 3-(128, 32, 64)-Dense 0 (0)—1772170692\validation	0,1591	4	14.27 min
model Conv 3-(64, 128, 32)-Dense 0 (0)—1772166734\validation	0,1741	4	12.07 min
model Conv 2-(32, 128)-Dense 0 (0)—1772152432\validation	0,1779	4	7.007 min
model Conv 3-(128, 32, 128)-Dense 0—1772218713\validation	0,1787	0	0
model Conv 3-(128, 64, 128)-Dense 0—1772221894\validation	0,2154	4	19.27 min
model Conv 2-(64, 64)-Dense 0 (0)—1772153466\validation	0,2412	4	8.301 min
model Conv 2-(32, 64)-Dense 0 (0)—1772152055\validation	0,2731	4	5.002 min

Fig. 13. Models loss leader-board

Despite these specific failure cases, the overall trend confirms that the most robust models require at least one dense layer to effectively classify the extracted features.

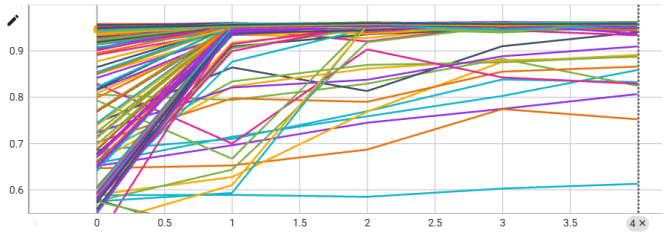

Fig. 14. Models accuracy with dense layer(s) on Validation

Run	Value	Step	Relative
model Conv 3-(64, 32, 64)-Dense 2-(128, 64)---1772780859\validation	0,9626	4	7.832 min
model Conv 3-(128, 32, 32)-Dense 1-(128,)---1772514496\validation	0,9624	4	14.9 min
model Conv 3-(32, 32, 64)-Dense 2-(64, 64)---1772645096\validation	0,9624	4	4.848 min
model Conv 3-(64, 32, 32)-Dense 2-(64, 64)---1772707289\validation	0,9624	4	7.053 min
model Conv 3-(32, 32, 32)-Dense 1-(64,)---1772467776\validation	0,9619	4	6.149 min
model Conv 3-(128, 64, 64)-Dense 1-(64,)---1772493460\validation	0,9619	4	18.01 min
model Conv 3-(128, 32, 32)-Dense 1-(128, 64)---1772844380\validation	0,9619	4	10.09 hr
model Conv 3-(64, 32, 64)-Dense 1-(64,)---1772481937\validation	0,9617	4	7.629 min
model Conv 3-(64, 32, 128)-Dense 1-(64,)---1772482574\validation	0,9615	4	8.559 min
model Conv 3-(64, 64, 64)-Dense 1-(64,)---1772483891\validation	0,9615	4	9.395 min
model Conv 3-(32, 64, 32)-Dense 2-(64, 128)---1772735510\validation	0,9615	4	5.892 min
model Conv 3-(64, 128, 128)-Dense 1-(128,)---1772513307\validation	0,9612	4	15.81 min
model Conv 3-(32, 32, 32)-Dense 2-(128, 128)---1772554879\validation	0,9612	4	5.414 min
model Conv 3-(128, 64, 64)-Dense 2-(128, 128)---1772591641\validation	0,9612	4	19.58 min
model Conv 3-(128, 128, 64)-Dense 2-(128, 128)---1772596253\validation	0,9612	4	23.46 min
model Conv 3-(64, 32, 128)-Dense 2-(64, 64)---1772708381\validation	0,9612	4	8.336 min
model Conv 3-(128, 32, 128)-Dense 1-(128, 64)---1772886061\validation	0,9612	4	53.1 min
model Conv 3-(64, 64, 32)-Dense 1-(128,)---1772508868\validation	0,961	4	9.346 min
model Conv 3-(128, 32, 64)-Dense 2-(128, 128)---1772587899\validation	0,961	4	15.25 min
model Conv 3-(128, 64, 128)-Dense 2-(64, 64)---1772723897\validation	0,961	4	20.83 min
model Conv 3-(64, 32, 32)-Dense 2-(64, 128)---1772756183\validation	0,961	4	7.065 min
model Conv 3-(64, 32, 128)-Dense 2-(128, 64)---1772781449\validation	0,961	4	9.05 min
model Conv 3-(32, 64, 128)-Dense 1-(64,)---1772470176\validation	0,9608	4	9.957 min
model Conv 3-(32, 64, 64)-Dense 1-(128,)---1772503777\validation	0,9608	4	6.263 min
model Conv 3-(64, 32, 32)-Dense 1-(128,)---1772506912\validation	0,9608	4	7.626 min
model Conv 3-(64, 128, 64)-Dense 1-(128,)---1772512162\validation	0,9608	4	14.45 min
model Conv 3-(128, 64, 64)-Dense 1-(128,)---1772519442\validation	0,9608	4	18.81 min
model Conv 3-(128, 128, 32)-Dense 1-(128,)---1772522521\validation	0,9608	4	24.76 min
model Conv 3-(64, 32, 64)-Dense 2-(128, 128)---1772561267\validation	0,9608	4	5.258 hr
model Conv 3-(64, 64, 128)-Dense 2-(128, 128)---1772582604\validation	0,9608	4	10.87 min
model Conv 3-(128, 32, 32)-Dense 2-(128, 128)---1772586681\validation	0,9608	4	16.77 min
model Conv 3-(128, 64, 32)-Dense 2-(64, 64)---1772720354\validation	0,9608	4	20.88 min

Fig. 15. Models with dense layers accuracy leader-board

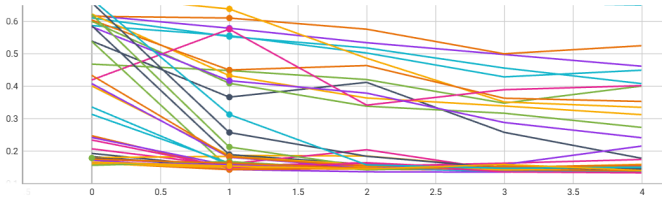

Fig. 16. Models with dense layer(s) Loss performance on Validation

C. Top Models

The 10 best-performing models on this static validation set were selected, as detailed in Fig. 18.

Interestingly, a closer observation of the top 10 best-performing models reveals a recurring pattern in their architectures. Within the convolutional layers, the depth tends to follow one of three specific trends: it either remains constant across all layers (e.g., 64-64-64), strictly decreases (e.g., 128-32-32), or decreases initially before increasing again in the final layer (e.g., 64-32-64).

A similar behavior appears in the fully connected classifier. For models utilizing two dense layers, the number of nodes

Run	Value	Step	Relative
model Conv 3-(32, 32, 128)-Dense 1-(128,)---1772502755\validation	0,1189	4	8.006 min
model Conv 3-(32, 32, 32)-Dense 2-(128, 128)---1772554879\validation	0,1201	4	5.414 min
model Conv 3-(32, 64, 64)-Dense 1-(128,)---1772503777\validation	0,1213	4	6.263 min
model Conv 3-(128, 32, 32)-Dense 1-(128, 64)---1772844380\validation	0,1217	4	10.09 hr
model Conv 3-(128, 32, 64)-Dense 1-(64,)---1772489690\validation	0,1219	4	14.79 min
model Conv 3-(128, 64, 64)-Dense 1-(128,)---1772519442\validation	0,1222	4	18.81 min
model Conv 3-(64, 64, 64)-Dense 1-(64,)---1772483891\validation	0,1222	4	9.395 min
model Conv 3-(128, 64, 128)-Dense 2-(64, 64)---1772723897\validation	0,1222	4	20.83 min
model Conv 3-(64, 64, 32)-Dense 1-(128,)---1772508868\validation	0,1231	4	9.346 min
model Conv 3-(64, 32, 128)-Dense 2-(64, 64)---1772708381\validation	0,1232	4	8.336 min
model Conv 3-(64, 128, 64)-Dense 1-(128,)---1772512162\validation	0,1232	4	14.45 min
model Conv 3-(64, 32, 128)-Dense 2-(128, 64)---1772781449\validation	0,1234	4	9.05 min
model Conv 3-(32, 64, 32)-Dense 2-(128, 64)---1772776826\validation	0,1235	4	5.455 min
model Conv 3-(32, 64, 32)-Dense 1-(64,)---1772469185\validation	0,1235	4	6.306 min
model Conv 3-(64, 128, 128)-Dense 1-(128,)---1772513307\validation	0,1237	4	15.81 min
model Conv 3-(128, 64, 64)-Dense 2-(128, 128)---1772591641\validation	0,1238	4	19.58 min
model Conv 3-(32, 32, 128)-Dense 2-(128, 64)---1772776359\validation	0,1238	4	6.174 min
model Conv 3-(32, 32, 128)-Dense 2-(64, 64)---1772645464\validation	0,1238	4	5.638 min
model Conv 3-(32, 128, 128)-Dense 2-(128, 128)---1772559911\validation	0,124	4	10.6 min
model Conv 3-(64, 32, 32)-Dense 2-(64, 64)---1772707289\validation	0,1241	4	7.053 min
model Conv 3-(128, 32, 128)-Dense 2-(64, 128)---1772765217\validation	0,1243	4	15.08 min
model Conv 3-(64, 128, 32)-Dense 1-(64,)---1772485392\validation	0,1243	4	12.6 min
model Conv 3-(64, 64, 128)-Dense 2-(64, 64)---1772710495\validation	0,1244	4	10.29 min
model Conv 3-(64, 32, 32)-Dense 1-(128,)---1772506912\validation	0,1244	4	7.626 min
model Conv 3-(64, 32, 128)-Dense 1-(128,)---1772508157\validation	0,1245	4	9.433 min
model Conv 3-(128, 64, 64)-Dense 1-(64,)---1772493460\validation	0,1245	4	18.01 min
model Conv 3-(64, 128, 64)-Dense 2-(64, 128)---1772761059\validation	0,1245	4	12.81 min
model Conv 3-(128, 32, 64)-Dense 2-(128, 128)---1772587899\validation	0,1246	4	15.25 min
model Conv 3-(32, 128, 128)-Dense 2-(64, 128)---1772755407\validation	0,1247	4	10.16 min
model Conv 3-(32, 32, 128)-Dense 2-(128, 128)---1772555729\validation	0,1247	4	7.417 min
model Conv 3-(32, 64, 64)-Dense 1-(64,)---1772469666\validation	0,1247	4	6.691 min
model Conv 3-(32, 64, 128)-Dense 1-(64,)---1772470176\validation	0,1251	4	9.957 min
model Conv 3-(64, 32, 64)-Dense 1-(64,)---1772481937\validation	0,1253	4	7.629 min

Fig. 17. Models dense layer loss leader-board

Fig. 18. Top 10 models

either remains the same (e.g., 64-64) or strictly decreases (e.g., 128-64).

From a learning perspective, these structural patterns suggest that reducing the complexity of the intermediate layers is an important factor in achieving high performance. It appears that by forcing the data through narrower layers, the network is encouraged to discard irrelevant noise and focus only on extracting the most important and discriminative details needed to accurately classify the cells.

However, evaluating performance on a single validation split may not provide a completely reliable measure of a model's true generalization capability. To obtain a more robust approximation of real-world performance, these top models were further evaluated using cross-validation on all the training data. The cross-validation results are presented in Fig. 19.

From these results, the best models can be identified based on key clinical metrics: accuracy, recall, precision, and F1-score. In this medical context, optimizing for **recall** ensures the model misses the fewest number of infected patients (minimizing false negatives). Conversely, optimizing for **precision** minimizes the number of healthy patients incorrectly diagnosed as infected (minimizing false positives). The best **accuracy** model maximizes overall correct predictions, while the highest **F1-score** represents the optimal harmonic balance

```
Different models results:
Model 1 - Acc: 0.9115 ± 0.1078, Loss: 0.2152 ± 0.1622, Recall: 0.9117 ± 0.1071, Precision: 0.9177 ± 0.1174
Model 2 - Acc: 0.9500 ± 0.0090, Loss: 0.1606 ± 0.0226, Recall: 0.9338 ± 0.0191, Precision: 0.9651 ± 0.0177
Model 3 - Acc: 0.9369 ± 0.0726, Loss: 0.1773 ± 0.1049, Recall: 0.9266 ± 0.0967, Precision: 0.9433 ± 0.0684
Model 4 - Acc: 0.9033 ± 0.1129, Loss: 0.2399 ± 0.1784, Recall: 0.8968 ± 0.1383, Precision: 0.9088 ± 0.1203
Model 5 - Acc: 0.9484 ± 0.0110, Loss: 0.1615 ± 0.0237, Recall: 0.9358 ± 0.0146, Precision: 0.9602 ± 0.0196
Model 6 - Acc: 0.9466 ± 0.0193, Loss: 0.1659 ± 0.0493, Recall: 0.9268 ± 0.0412, Precision: 0.9649 ± 0.0115
Model 7 - Acc: 0.9192 ± 0.1041, Loss: 0.2034 ± 0.1437, Recall: 0.9153 ± 0.1274, Precision: 0.9260 ± 0.1033
Model 8 - Acc: 0.8590 ± 0.1805, Loss: 0.2625 ± 0.2163, Recall: 0.7460 ± 0.3733, Precision: 0.7714 ± 0.3860
Model 9 - Acc: 0.9493 ± 0.0089, Loss: 0.1595 ± 0.0196, Recall: 0.9376 ± 0.0119, Precision: 0.9600 ± 0.0169
Model 10 - Acc: 0.9500 ± 0.0094, Loss: 0.1608 ± 0.0256, Recall: 0.9405 ± 0.0133, Precision: 0.9587 ± 0.0173

Summary of the best models:
Best Accuracy: Model 10 (0.9500 ± 0.0094)
Best Recall: Model 10 (0.9405 ± 0.0133)
Best Precision: Model 2 (0.9651 ± 0.0177)
Best F1-score: Model 10 (0.9495)
```

Fig. 19. Top 10 models Cross-validation Results

between precision and recall.

As shown in Fig. 19, Model 10 achieved the highest scores in Accuracy, Recall, and F1-score. Meanwhile, Model 2 achieved the highest Precision, while matching Model 10 in overall Accuracy.

D. Test Validation

Having identified the optimal architectures for these target metrics, the final step is to evaluate their definitive predictive power. To accomplish this, Model 2 and Model 10 were retrained using the entire aggregated dataset (combining both training and validation data). They were then evaluated on the holdout test set (data that remained entirely unseen by any model during the hyperparameter tuning phase).

```
Test results for Model 2:
Precision: 0.9712308645248413
Accuracy: 0.9617198705673218
Recall: 0.9528403282165527
F1-Score: 0.9619477067367317
------------------------------
```

Fig. 20. Results Model 2

```
Test results for Model 10:
Precision: 0.9657410979270935
Accuracy: 0.9608127474784851
Recall: 0.9567703008651733
F1-Score: 0.9612347696892988
------------------------------
```

Fig. 21. Results Model 10

Surprisingly, while the results on the test set are highly similar, Model 2 slightly outperforms Model 10 across all metrics except Recall (see Fig. 20 and Fig. 21). Despite these minor differences, both models demonstrate strong reliability and robustness in successfully identifying malaria infections in unseen cell images.

V. Conclusion

This project successfully modernized and optimized a convolutional neural network for the automated binary classification of malaria in blood cell images. Through systematic experimentation, several key insights were derived regarding both data processing and model architecture.

First, it was demonstrated that increasing the input resolution to 128×128 pixels, when combined with a dynamic and extensive data augmentation pipeline, significantly improved the model's ability to retain crucial morphological details under perturbation.

Secondly, the architectural analysis revealed that a three-layer convolutional base provided the optimal feature extraction foundation. Observing the top-performing models highlighted the effectiveness of an "information bottleneck" (design where progressively narrowing the layer depths forces the network to discard irrelevant noise and isolate the most discriminative features). It was also found that while fully connected dense layers are essential for final classification, their size must be carefully tuned to prevent performance degradation.

Finally, rigorous evaluation through cross-validation allowed for the identification of specialized models tailored to specific clinical metrics, such as precision (Model 2) and recall (Model 10). The ultimate evaluation on the unseen test set confirmed that both selected architectures are highly reliable and robust in identifying malaria infections.

Overall, this study highlights the importance of balancing input data resolution, architectural complexity, and targeted evaluation metrics when developing deep learning solutions for critical biomedical applications.

References

[1] Malaria cell images dataset. [Online]. Available: https://www.kaggle.com/datasets/iarunava/cell-images-for-detecting-malaria

[2] M. Poostchi, K. Silamut, R. J. Maude, S. Jaeger, and G. Thoma, "Image analysis and machine learning for detecting malaria," vol. 194, pp. 36–55. [Online]. Available: https://linkinghub.elsevier.com/retrieve/pii/S193152441730333X

[3] M. C. S. . Malaria-malaria - symptoms & causes. [Online]. Available: https://www.mayoclinic.org/diseases-conditions/malaria/symptoms-causes/syc-20351184

[4] Detecting malaria | CNN. [Online]. Available: https://kaggle.com/code/kushal1996/detecting-malaria-cnn

[5] Albumentations: fast and flexible image augmentations. [Online]. Available: https://albumentations.ai/

[6] What is adam optimizer? Section: Deep Learning. [Online]. Available: https://www.geeksforgeeks.org/deep-learning/adam-optimizer/

[7] Binary cross entropy/log loss for binary classification. Section: Blogathon. [Online]. Available: https://www.geeksforgeeks.org/deep-learning/binary-cross-entropy-log-loss-for-binary-classification/

[8] . v. D. m. l. . sept. 2018. Deep learning basics with python, TensorFlow and keras. [Online]. Available: http://www.youtube.com/playlist?list=PLQVvvaa0QuDfhTox0AjmQ6tvTgMBZBEXN